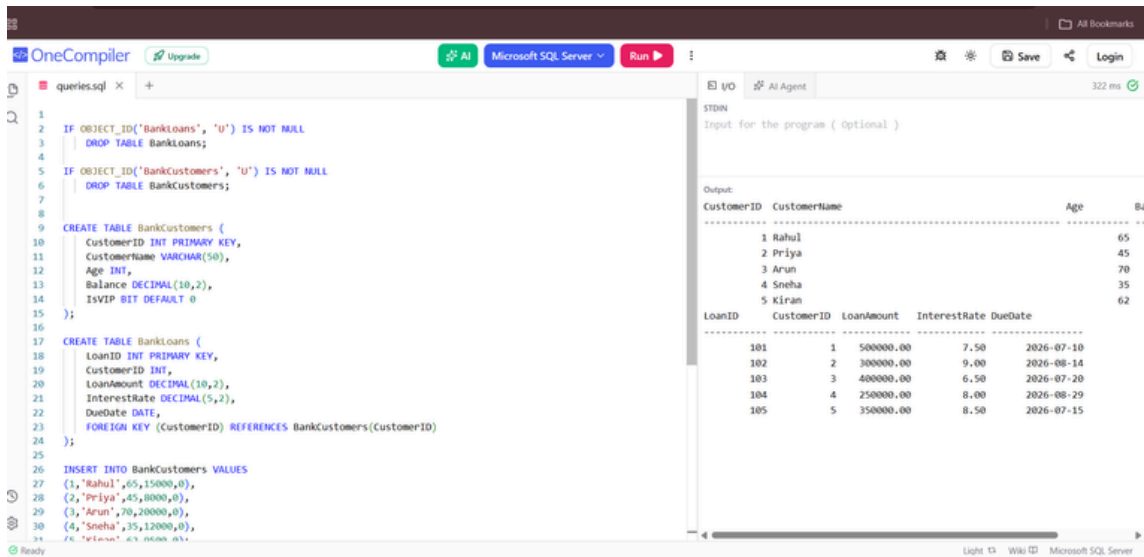


WEEK 1 ASSIGNMENT

PLSQL

Exercise 1: Control Structures



The screenshot shows the OneCompiler IDE with a SQL script in the editor and its output in the right-hand pane. The script creates two tables, BankCustomers and BankLoans, and inserts data into them. The output displays the data for both tables.

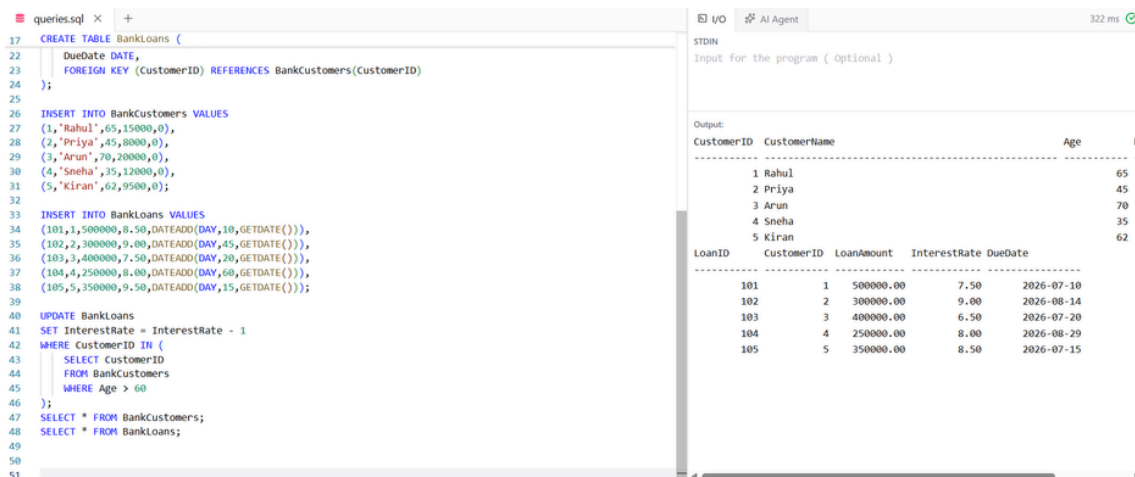
```
1 IF OBJECT_ID('BankLoans', 'U') IS NOT NULL
2   DROP TABLE BankLoans;
3
4 IF OBJECT_ID('BankCustomers', 'U') IS NOT NULL
5   DROP TABLE BankCustomers;
6
7
8 CREATE TABLE BankCustomers (
9   CustomerID INT PRIMARY KEY,
10  CustomerName VARCHAR(50),
11  Age INT,
12  Balance DECIMAL(10,2),
13  IsVIP BIT DEFAULT 0
14 );
15
16 CREATE TABLE BankLoans (
17   LoanID INT PRIMARY KEY,
18   CustomerID INT,
19   LoanAmount DECIMAL(10,2),
20   InterestRate DECIMAL(5,2),
21   DueDate DATE,
22   FOREIGN KEY (CustomerID) REFERENCES BankCustomers(CustomerID)
23 );
24
25 INSERT INTO BankCustomers VALUES
26 (1, 'Rahul', 65, 15000, 0),
27 (2, 'Priya', 45, 8000, 0),
28 (3, 'Arun', 70, 20000, 0),
29 (4, 'Sneha', 35, 12000, 0),
30 (5, 'Kiran', 62, 9500, 0);
31
```

Output:

CustomerID	CustomerName	Age	Balance
1	Rahul	65	15000.00
2	Priya	45	8000.00
3	Arun	70	20000.00
4	Sneha	35	12000.00
5	Kiran	62	9500.00

LoanID	CustomerID	LoanAmount	InterestRate	DueDate
101	1	500000.00	7.50	2026-07-10
102	2	300000.00	9.00	2026-08-14
103	3	400000.00	6.50	2026-07-20
104	4	250000.00	8.00	2026-08-29
105	5	350000.00	8.50	2026-07-15

SCENARIO 1



The screenshot shows the OneCompiler IDE with a SQL script in the editor and its output in the right-hand pane. The script creates two tables, BankCustomers and BankLoans, and inserts data into them. It then updates the interest rate for loans based on the customer's age and displays the results.

```
17 CREATE TABLE BankLoans (
18   DueDate DATE,
19   FOREIGN KEY (CustomerID) REFERENCES BankCustomers(CustomerID)
20 );
21
22 INSERT INTO BankCustomers VALUES
23 (1, 'Rahul', 65, 15000, 0),
24 (2, 'Priya', 45, 8000, 0),
25 (3, 'Arun', 70, 20000, 0),
26 (4, 'Sneha', 35, 12000, 0),
27 (5, 'Kiran', 62, 9500, 0);
28
29 INSERT INTO BankLoans VALUES
30 (101, 1, 500000, 8.50, DATEADD(DAY, 10, GETDATE())),
31 (102, 2, 300000, 9.00, DATEADD(DAY, 45, GETDATE())),
32 (103, 3, 400000, 7.50, DATEADD(DAY, 20, GETDATE())),
33 (104, 4, 250000, 8.00, DATEADD(DAY, 60, GETDATE())),
34 (105, 5, 350000, 8.50, DATEADD(DAY, 15, GETDATE()));
35
36 UPDATE BankLoans
37 SET InterestRate = InterestRate - 1
38 WHERE CustomerID IN (
39   SELECT CustomerID
40   FROM BankCustomers
41   WHERE Age > 60
42 );
43
44 SELECT * FROM BankCustomers;
45
46 SELECT * FROM BankLoans;
47
```

Output:

CustomerID	CustomerName	Age	Balance
1	Rahul	65	15000.00
2	Priya	45	8000.00
3	Arun	70	20000.00
4	Sneha	35	12000.00
5	Kiran	62	9500.00

LoanID	CustomerID	LoanAmount	InterestRate	DueDate
101	1	500000.00	7.50	2026-07-10
102	2	300000.00	9.00	2026-08-14
103	3	400000.00	6.50	2026-07-20
104	4	250000.00	8.00	2026-08-29
105	5	350000.00	8.50	2026-07-15

SCENARIO 2

queries.sql

```
17 CREATE TABLE BankLoans (  
19     CustomerID INT,  
20     LoanAmount DECIMAL(10,2),  
21     InterestRate DECIMAL(5,2),  
22     DueDate DATE,  
23     FOREIGN KEY (CustomerID) REFERENCES BankCustomers(CustomerID)  
24 );  
25  
26 INSERT INTO BankCustomers VALUES  
27 (1,'Rahul',65,15000,0),  
28 (2,'Priya',45,8000,0),  
29 (3,'Arun',70,20000,0),  
30 (4,'Sneha',35,12000,0),  
31 (5,'Kiran',62,9500,0);  
32  
33 INSERT INTO BankLoans VALUES  
34 (101,1,500000,8.50,DATEADD(DAY,10,GETDATE())),  
35 (102,2,300000,9.00,DATEADD(DAY,45,GETDATE())),  
36 (103,3,400000,7.50,DATEADD(DAY,20,GETDATE())),  
37 (104,4,250000,8.00,DATEADD(DAY,60,GETDATE())),  
38 (105,5,350000,9.50,DATEADD(DAY,15,GETDATE()));  
39  
40  
41 UPDATE BankCustomers  
42 SET IsVIP = 1  
43 WHERE Balance > 10000;  
44 SELECT * FROM BankCustomers;  
45 SELECT * FROM BankLoans;  
46  
47  
48
```

I/O

AI Agent

331 ms

STDB

Input for the program (optional)

Output

CustomerID	CustomerName	Age	B
1	Rahul	65	
2	Priya	45	
3	Arun	70	
4	Sneha	35	
5	Kiran	62	

LoanID	CustomerID	LoanAmount	InterestRate	DueDate
101	1	500000.00	8.50	2026-07-10
102	2	300000.00	9.00	2026-08-14
103	3	400000.00	7.50	2026-07-20
104	4	250000.00	8.00	2026-08-29
105	5	350000.00	9.50	2026-07-15

SCENARIO 3

queries.sql

```
26 INSERT INTO BankCustomers VALUES  
27 (1,'Rahul',65,15000,0),  
28 (2,'Priya',45,8000,0),  
29 (3,'Arun',70,20000,0),  
30 (4,'Sneha',35,12000,0),  
31 (5,'Kiran',62,9500,0);  
32  
33 INSERT INTO BankLoans VALUES  
34 (101,1,500000,8.50,DATEADD(DAY,10,GETDATE())),  
35 (102,2,300000,9.00,DATEADD(DAY,45,GETDATE())),  
36 (103,3,400000,7.50,DATEADD(DAY,20,GETDATE())),  
37 (104,4,250000,8.00,DATEADD(DAY,60,GETDATE())),  
38 (105,5,350000,9.50,DATEADD(DAY,15,GETDATE()));  
39  
40  
41 SELECT  
42     b.CustomerName,  
43     l.LoanID,  
44     l.DueDate,  
45     'Reminder: Loan due within 30 days' AS Reminder  
46 FROM BankCustomers b  
47 JOIN BankLoans l  
48 ON b.CustomerID = l.CustomerID  
49 WHERE l.DueDate BETWEEN CAST(GETDATE() AS DATE)  
50     AND DATEADD(DAY,30,CAST(GETDATE() AS DATE));  
51  
52 SELECT * FROM BankCustomers;  
53 SELECT * FROM BankLoans;  
54  
55  
56
```

I/O

AI Agent

324 ms

STDB

Input for the program (optional)

Output

CustomerName	LoanID	DueDate
Rahul	101	2026-07-10
Arun	103	2026-07-20
Kiran	105	2026-07-15

CustomerID	CustomerName	Age	B
1	Rahul	65	
2	Priya	45	
3	Arun	70	
4	Sneha	35	
5	Kiran	62	

LoanID	CustomerID	LoanAmount	InterestRate	DueDate
101	1	500000.00	8.50	2026-07-10
102	2	300000.00	9.00	2026-08-14
103	3	400000.00	7.50	2026-07-20
104	4	250000.00	8.00	2026-08-29
105	5	350000.00	9.50	2026-07-15

Exercise 3: Stored Procedures

SCENARIO 1

```
1 CREATE TABLE Accounts (
2   AccountID INT PRIMARY KEY,
3   CustomerName VARCHAR(50),
4   AccountType VARCHAR(20),
5   Balance DECIMAL(10,2)
6 );
7
8 INSERT INTO Accounts VALUES
9 (101,'Rahul','Savings',10000),
10 (102,'Priya','Savings',20000);
11
12 GO
13
14 CREATE PROCEDURE ProcessMonthlyInterest
15 AS
16 BEGIN
17   UPDATE Accounts
18   SET Balance = Balance + (Balance * 0.01)
19   WHERE AccountType='Savings';
20 END;
21
22 GO
23 EXEC ProcessMonthlyInterest;
24
25 SELECT * FROM Accounts;
```

Output:

AccountID	CustomerName	AccountType
101	Rahul	Savings
102	Priya	Savings

SCENARIO 2

```
1 CREATE TABLE Employees (
2   EmployeeID INT PRIMARY KEY,
3   EmployeeName VARCHAR(50),
4   Department VARCHAR(50),
5   Salary DECIMAL(10,2)
6 );
7
8
9
10 INSERT INTO Employees VALUES
11 (1,'Amit','HR',40000),
12 (2,'Priya','IT',50000),
13 (3,'Rahul','IT',60000),
14 (4,'Sneha','Finance',55000);
15
16
17
18 UPDATE Employees
19 SET Salary = Salary + (Salary * 10 / 100)
20 WHERE Department = 'IT';
21
22 SELECT * FROM Employees;
```

Output:

EmployeeID	EmployeeName	Department
1	Amit	HR
2	Priya	IT
3	Rahul	IT
4	Sneha	Finance

SCENARIO 3

```
9 INSERT INTO Accounts VALUES
10 (101,'Rahul','Savings',10000),
11 (102,'Priya','Savings',20000),
12 (103,'Arun','Current',15000);
13
14
15 DECLARE @Amount DECIMAL(10,2) = 2000;
16 DECLARE @Balance DECIMAL(10,2);
17
18 SELECT @Balance = Balance
19 FROM Accounts
20 WHERE AccountID = 101;
21
22 IF @Balance >= @Amount
23 BEGIN
24   UPDATE Accounts
25   SET Balance = Balance - @Amount
26   WHERE AccountID = 101;
27
28   UPDATE Accounts
29   SET Balance = Balance + @Amount
30   WHERE AccountID = 102;
31   PRINT 'Transfer Successful';
32 END
33 ELSE
34 BEGIN
35   PRINT 'Insufficient Balance';
36 END;
37
38
39 SELECT * FROM Accounts;
```

Output:

AccountID	CustomerName	AccountType
101	Rahul	Savings
102	Priya	Savings
103	Arun	Current